

United International University

CSE 3422: Software Engineering Lab

GIT

25/03/26

Contents

1 Lab Overview **1**

2 Learning Objectives **1**

3 What is Git? **1**

4 What is GitHub? **1**

5 Step 1: Install Git **1**

6 Step 2: Configure Git **2**

7 Step 3: Create GitHub Account **2**

8 Step 4: Create Local Repository **2**

8.1 Explanation 2

9 Step 5: Create and Track Files **2**

10 Git Workflow **3**

11 Step 6: Create GitHub Repository **3**

12 Step 7: Connect to GitHub **3**

13 Step 8: Making Changes **3**

14 Step 9: Viewing History **3**

15 Step 10: Branching **4**

16 Lab Tasks **4**

16.1 Task 1: Setup 4

16.2 Task 2: Local Repository 4

16.3 Task 3: GitHub Integration 4

16.4 Task 4: Update Project 4

16.5 Task 5: Branching 4

17 Common Errors **5**

18 Scenario **5**

19 Step 1: Initial Setup **5**

20 Step 2: Create GitHub Repository **5**

21 Step 3: Feature Development Using Branches **6**

21.1 Create a branch 6

21.2 Modify file 6

21.3 Commit changes 6

21.4 Push branch 6

22 Step 4: Switching Contexts **6**

23 Step 5: Merging Branches **7**

24 Step 6: Merge Conflict Scenario **7**

24.1 Create conflict 7

24.2 What happens? 7

24.3 Fix conflict 8

25 Step 7: Pull Changes (Collaboration) **8**

26 Step 8: Mistake Recovery **8**

27 Lab Tasks **8**

27.1 Task 1 8

27.2 Task 2 8

27.3 Task 3 9

27.4 Task 4 9

27.5 Task 5 9

28 Key Takeaways **9**

29 Conclusion **9**

1 Lab Overview

In this lab, students will learn how to use Git and GitHub for version control and collaboration. This includes installing Git, setting up GitHub, and performing basic version control operations.

2 Learning Objectives

By the end of this lab, students will be able to:

- Understand version control systems
- Install and configure Git
- Create and manage repositories
- Track changes using Git
- Push code to GitHub
- Perform basic branching operations

3 What is Git?

Git is a distributed version control system that allows developers to:

- Track changes in files
- Revert to previous versions
- Work collaboratively

4 What is GitHub?

GitHub is a cloud-based platform that hosts Git repositories and enables collaboration.

5 Step 1: Install Git

Download Git from:

`https://git-scm.com`

Verify installation:

```
git --version
```

6 Step 2: Configure Git

Set your identity:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

Verify configuration:

```
git config --list
```

7 Step 3: Create GitHub Account

Go to:

`https://github.com`

Create an account using your email.

8 Step 4: Create Local Repository

```
mkdir git-lab
cd git-lab
git init
```

8.1 Explanation

- `git init` creates a new Git repository
- A hidden folder `.git` is created

9 Step 5: Create and Track Files

Create a file:

```
echo Hello Git > file.txt
```

Check status:

```
git status
```

Add file:

```
git add file.txt
```

Commit file:

```
git commit -m "First commit"
```

10 Git Workflow

`Working Directory → Staging Area → Repository`

11 Step 6: Create GitHub Repository

- Go to GitHub
- Click "New Repository"
- Name it: `git-lab`

12 Step 7: Connect to GitHub

```
git remote add origin https://github.com/YOUR_USERNAME/git-lab.
    git
git branch -M main
git push -u origin main
```

13 Step 8: Making Changes

Edit file and add new text.

Check status:

```
git status
```

Commit changes:

```
git add file.txt
git commit -m "Updated file"
git push
```

14 Step 9: Viewing History

```
git log
```

15 Step 10: Branching

Create branch:

```
git branch feature1
```

Switch branch:

```
git checkout feature1
```

Merge branch:

```
git checkout main
git merge feature1
```

16 Lab Tasks

16.1 Task 1: Setup

- Install Git
- Create GitHub account
- Configure Git

16.2 Task 2: Local Repository

- Create a repository
- Add two files
- Commit them

16.3 Task 3: GitHub Integration

- Create repository on GitHub
- Push code

16.4 Task 4: Update Project

- Modify a file
- Commit and push changes

16.5 Task 5: Branching

- Create a branch
- Add a new file
- Merge into main

17 Common Errors

Git not recognized

Git is not installed or not added to PATH.

Authentication failed

Check GitHub credentials.

Nothing to commit

Make sure files are modified and added.

18 Scenario

You are building a project called:

Student Management System

Multiple developers are working on different features.

19 Step 1: Initial Setup

```
mkdir git-lab
cd git-lab
git init
```

Create a file:

```
echo "Student System" > app.txt
```

Commit it:

```
git add .
git commit -m "Initial commit"
```

20 Step 2: Create GitHub Repository

Create a repo on GitHub and connect:

```
git remote add origin https://github.com/YOUR_USERNAME/git-lab.
    git
git branch -M main
git push -u origin main
```

21 Step 3: Feature Development Using Branches

Problem: You want to add a login feature without breaking main code.

21.1 Create a branch

```
git checkout -b feature-login
```

21.2 Modify file

```
echo "Login Feature Added" >> app.txt
```

21.3 Commit changes

```
git add .
git commit -m "Added login feature"
```

21.4 Push branch

```
git push origin feature-login
```

22 Step 4: Switching Contexts

Problem: You now need to work on another feature.

```
git checkout main
git checkout -b feature-dashboard
```

Add:

```
echo "Dashboard Feature" >> app.txt
git add .
git commit -m "Added dashboard"
git push origin feature-dashboard
```

23 Step 5: Merging Branches

Problem: You want to combine features into main.

```
git checkout main
git merge feature-login
git merge feature-dashboard
```

Push:

```
git push
```

24 Step 6: Merge Conflict Scenario

Problem: Two developers edit the same line.

24.1 Create conflict

```
git checkout -b conflict-branch
```

Edit:

```
echo "Conflicting Change A" > app.txt
git commit -am "Change A"
```

Go back:

```
git checkout main
echo "Conflicting Change B" > app.txt
git commit -am "Change B"
```

Now merge:

```
git merge conflict-branch
```

24.2 What happens?

Git will show a conflict.

File will look like:

```
<<<<<<< HEAD
Conflicting Change B
=======
Conflicting Change A
>>>>>>> conflict-branch
```

24.3 Fix conflict

Edit file manually:

```
Final Correct Version
```

Then:

```
git add app.txt
git commit -m "Resolved conflict"
```

25 Step 7: Pull Changes (Collaboration)

Problem: Your teammate pushed new code.

```
git pull origin main
```

26 Step 8: Mistake Recovery

Problem: You added a wrong file.

```
git reset HEAD file.txt
```

Problem: You want to undo last commit.

```
git reset --soft HEAD~1
```

27 Lab Tasks

27.1 Task 1

Create a repository and push initial code.

27.2 Task 2

Create two branches:

- feature-login
- feature-profile

Add different content in each.

27.3 Task 3

Merge both branches into main.

27.4 Task 4

Create a merge conflict and resolve it.

27.5 Task 5

Push all branches to GitHub.

28 Key Takeaways

- Branches allow parallel development
- Conflicts are normal and must be resolved manually
- Git tracks changes, not files
- Always commit before switching tasks

29 Conclusion

You now understand real-world Git workflows including:

- Branching
- Merging
- Conflict resolution
- Collaboration